

Homework 06

Haoran Xu

2026-10-04

All work is in R with base functions only (no pROC, class, or caret), so every step of the KNN, ROC, and AUC computation is visible. One seed, 43261, fixes the single 70/30 split used by every model in Questions 1 and 2.

```
# KNN class probabilities: fraction of the k nearest training points with y = 1.
# Euclidean distance, equal weights. Distance ties are broken by training row number.
knn_prob <- function(Xtr, ytr, Xte, ks) {
  D <- outer(rowSums(Xte^2), rowSums(Xtr^2), "+") - 2 * Xte %*% t(Xtr) # squared distances
  nb <- t(apply(D, 1, order))[, 1:max(ks), drop = FALSE] # neighbours sorted once, reused
  for all k
  Y <- matrix(ytr[nb], nrow(Xte))
  sapply(ks, function(k) rowMeans(Y[, 1:k, drop = FALSE]))
}
# ROC curve: classify y = 1 when score >= c, for c sweeping over all distinct scores.
roc <- function(score, y) {
  cut <- c(Inf, sort(unique(score), decreasing = TRUE)) # Inf gives the (0, 0) corner
  tpr <- sapply(cut, function(c) mean(score[y == 1] >= c))
  fpr <- sapply(cut, function(c) mean(score[y == 0] >= c))
  list(fpr = fpr, tpr = tpr,
       auc = sum(diff(fpr) * (head(tpr, -1) + tail(tpr, -1)) / 2)) # trapezoid rule
}
```

Question 1 (Compare KNN and logistic regression)

Solution

Seed 43261; 398 training and 171 test observations ($0.7 \times 569 = 398.3$). The three predictors are centered and scaled with the training means and standard deviations only.

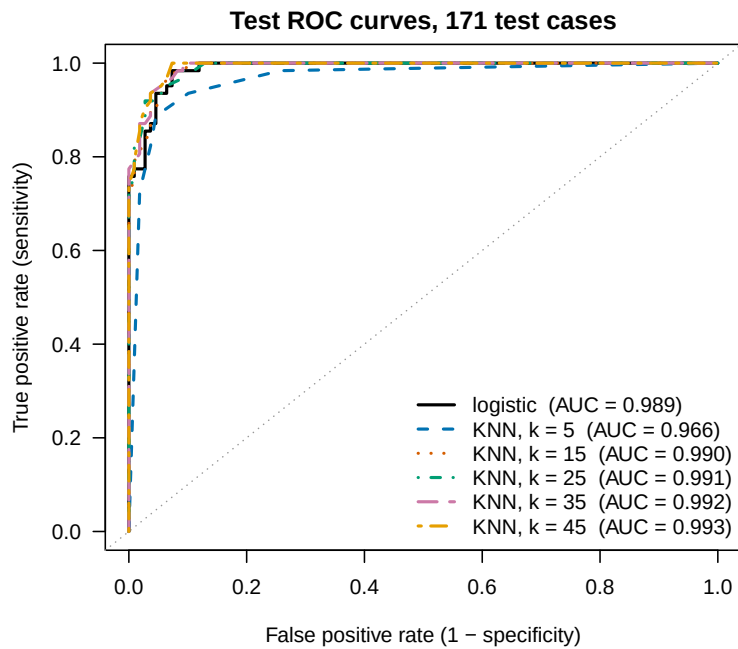
```
bc <- read.csv("data/breast-cancer.csv")
set.seed(43261)
tr <- sort(sample(nrow(bc), round(0.7 * nrow(bc))))
xv <- c("mean_radius", "mean_texture", "mean_smoothness")
mu <- colMeans(bc[tr, xv]); sdv <- apply(bc[tr, xv], 2, sd) # training statistics
only
Xtr <- as.matrix(sweep(sweep(bc[tr, xv], 2, mu), 2, sdv, "/"))
Xte <- as.matrix(sweep(sweep(bc[-tr, xv], 2, mu), 2, sdv, "/"))
ytr <- bc$y[tr]; yte <- bc$y[-tr]
c(n_train = length(tr), n_test = length(yte), malignant_train = mean(ytr), malignant_test =
  mean(yte))
#>      n_train      n_test malignant_train malignant_test
#>    398.000000    171.000000      0.376884      0.362573
round(rbind(mean = mu, sd = sdv), 4)
#>      mean_radius mean_texture mean_smoothness
#> mean      14.1154      19.2534      0.0958
#> sd         3.4501       4.1884      0.0137
```

```

ks1 <- c(5, 15, 25, 35, 45)
glm1 <- glm(ytr ~ Xtr, family = binomial) # intercept, no penalty
p_lr <- predict(glm1, newdata = list(Xtr = Xte), type = "response")
round(coef(glm1), 3)
#>      (Intercept)      Xtrmean_radius      Xtrmean_texture Xtrmean_smoothness
#>      -1.008      4.554      1.611      1.960
P1 <- cbind(logistic = as.vector(p_lr), knn_prob(Xtr, ytr, Xte, ks1))
colnames(P1)[-1] <- paste0("knn", ks1)
R1 <- apply(P1, 2, roc, y = yte)
auc1 <- sapply(R1, `[[`, "auc"); round(auc1, 4)
#> logistic      knn5      knn15      knn25      knn35      knn45
#> 0.9888 0.9657 0.9899 0.9906 0.9917 0.9927

par(mar = c(4, 4, 2, 1), las = 1, cex = 0.8)
cols <- c("#000000", "#0072B2", "#D55E00", "#009E73", "#CC79A7", "#E69F00")
ltys <- c(1, 2, 3, 4, 5, 6)
plot(NA, xlim = c(0, 1), ylim = c(0, 1), xlab = "False positive rate (1 - specificity)",
     ylab = "True positive rate (sensitivity)", main = "Test ROC curves, 171 test cases")
abline(0, 1, lty = 3, col = "gray60")
for (j in seq_along(R1)) lines(R1[[j]]$fpr, R1[[j]]$tpr, col = cols[j], lty = ltys[j], lwd =
  ↪ 2)
legend("bottomright", bty = "n", col = cols, lty = ltys, lwd = 2,
     legend = sprintf("%s (AUC = %.3f)", c("logistic", paste0("KNN, k = ", ks1)), auc1))

```



model	logistic	$k = 5$	$k = 15$	$k = 25$	$k = 35$	$k = 45$
test AUC	0.989	0.966	0.990	0.991	0.992	0.993

Preference. Based on these results I prefer **KNN with $k = 45$** , with logistic regression as a close runner-up.

- It has the highest test AUC (0.993), and test AUC increases monotonically in k over the whole grid, so the largest k tried is the best supported choice. Its curve is also on or above the others in the low false-positive region (left edge of the plot), which is the region that matters for a cancer screen.

- Logistic regression (0.989) sits between $k = 5$ and $k = 15$. Its curve dips below the KNN curves around a false positive rate of 0.02 to 0.05, where it catches about 5 percentage points fewer malignant cases at the same specificity.
- $k = 5$ is clearly worst (0.966): its probabilities take only six distinct values (0, 0.2, ..., 1) and are noisy, so the curve is coarse and lower.
- With 398 training cases and only 3 standardized predictors, the class boundary is smooth, so averaging over 45 neighbours (about 11% of the training set) reduces variance while adding little bias. The monotone trend suggests even larger k might do as well.
- Caveat: the five curves for logistic and $k \geq 15$ differ by at most 0.004 in AUC on 171 test cases, which is within split-to-split noise. If I had to pick a model for deployment I would weigh that against the interpretability of three logistic coefficients (radius, texture, and smoothness all raise the malignancy odds) and would confirm the ranking by cross-validation before choosing.

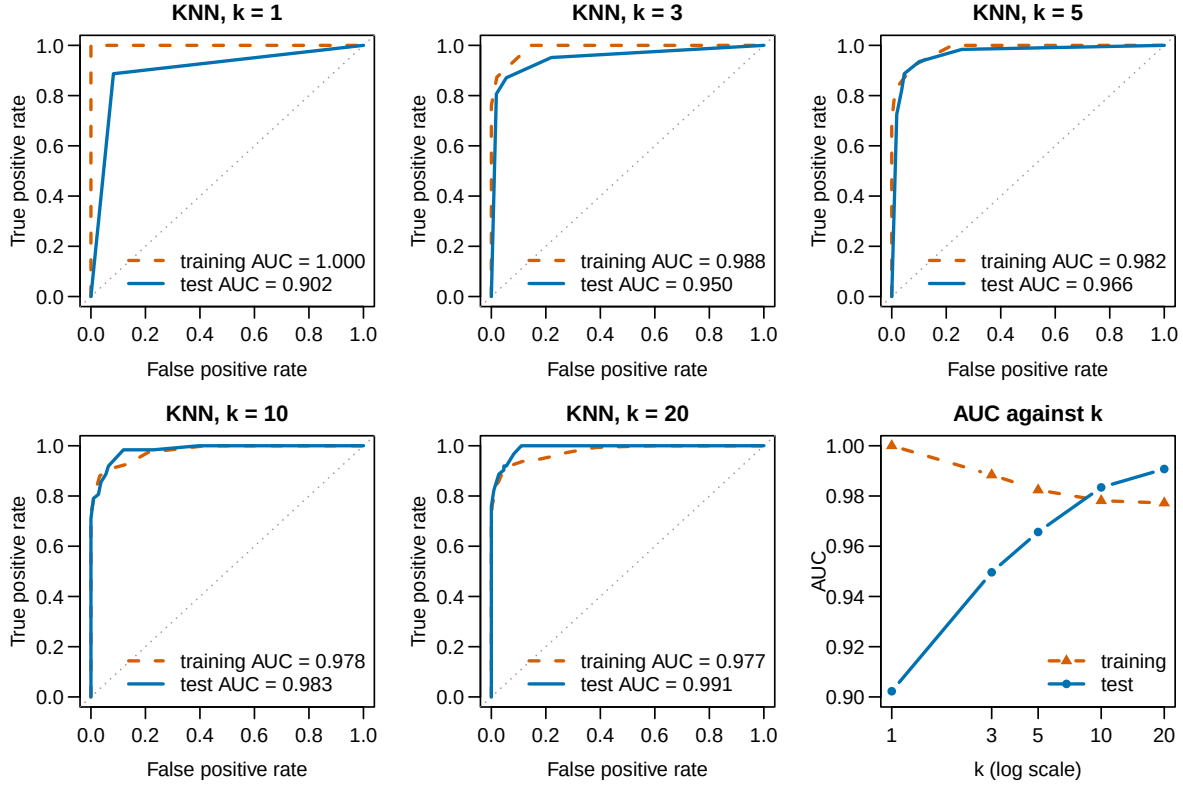
Question 2 (Training and test ROC curves)

Solution

Same split and standardization. For training predictions the query point is itself one of its k nearest neighbours (distance zero, ranked first), exactly as the question asks.

```
ks2 <- c(1, 3, 5, 10, 20)
Ptr <- knn_prob(Xtr, ytr, Xtr, ks2)      # training rows as queries: each one finds itself
  ⇨ first
Pte <- knn_prob(Xtr, ytr, Xte, ks2)
Rtr <- apply(Ptr, 2, roc, y = ytr); Rte <- apply(Pte, 2, roc, y = yte)
auc2 <- rbind(train = sapply(Rtr, `[`, "auc"), test = sapply(Rte, `[`, "auc"))
colnames(auc2) <- paste0("k", ks2); round(auc2, 4)
#>           k1      k3      k5     k10     k20
#> train 1.0000 0.9884 0.9824 0.9781 0.9772
#> test  0.9023 0.9496 0.9657 0.9834 0.9908
c(train_error_k1 = mean((Ptr[, 1] > 0.5) != ytr), test_error_k1 = mean((Pte[, 1] > 0.5) !=
  ⇨ yte))
#> train_error_k1 test_error_k1
#>      0.0000000      0.0935673
```

```
par(mfrow = c(2, 3), mar = c(3.6, 3.8, 2, 0.8), mgp = c(2.2, 0.7, 0), las = 1, cex = 0.72)
for (j in seq_along(ks2)) {
  plot(NA, xlim = c(0, 1), ylim = c(0, 1), xlab = "False positive rate",
    ylab = "True positive rate", main = sprintf("KNN, k = %d", ks2[j]))
  abline(0, 1, lty = 3, col = "gray60")
  lines(Rtr[[j]]$fpr, Rtr[[j]]$tpr, col = "#D55E00", lty = 2, lwd = 2)
  lines(Rte[[j]]$fpr, Rte[[j]]$tpr, col = "#0072B2", lty = 1, lwd = 2)
  legend("bottomright", bty = "n", col = c("#D55E00", "#0072B2"), lty = c(2, 1), lwd = 2,
    legend = sprintf("%s AUC = %.3f", c("training", "test"), auc2[, j]))
}
plot(ks2, auc2["train", ], type = "b", pch = 17, lty = 2, lwd = 2, col = "#D55E00", log =
  ⇨ "x",
  ylim = c(0.9, 1), xaxt = "n", xlab = "k (log scale)", ylab = "AUC", main = "AUC against
  ⇨ k")
axis(1, at = ks2)
lines(ks2, auc2["test", ], type = "b", pch = 16, lwd = 2, col = "#0072B2")
legend("bottomright", bty = "n", c("training", "test"), col = c("#D55E00", "#0072B2"),
  pch = c(17, 16), lty = c(2, 1), lwd = 2)
```



	$k = 1$	$k = 3$	$k = 5$	$k = 10$	$k = 20$
training AUC	1.000	0.988	0.982	0.978	0.977
test AUC	0.902	0.950	0.966	0.983	0.991

What happens at $k = 1$. Each training point is its own nearest neighbour, so the estimated probability is its own label: $\hat{p}_i = y_i$ exactly. The training ROC curve jumps from $(0, 0)$ straight to $(0, 1)$ and the training AUC is exactly 1 (training error 0). This is pure memorization: the model has not learned anything about new patients, it has looked up the answer. On the test set the same rule gives probabilities in $\{0, 1\}$ from the single nearest *other* patient, the ROC curve is one point plus the two corners, and the AUC is 0.902, the worst of the five. With a single interior point the trapezoid area is (sensitivity + specificity)/2: here $(0.887 + 0.917)/2$, so 1NN gets no credit for ranking, only for its one hard classification.

How the gap changes with k .

- **Training AUC falls** as k grows (1.000 to 0.977): the point's own label is diluted by $k - 1$ other labels, so the self-vote advantage shrinks from a full lookup at $k = 1$ to a weight of $1/k$. The training curve approaches what an honest estimate would give.
- **Test AUC rises** steadily (0.902 to 0.991): averaging more neighbours reduces the variance of $\hat{p}$, and with three predictors and 398 training cases, 20 neighbours are still local enough that the extra bias is small.
- **The gap closes and then reverses.** Training minus test AUC is 0.098 at $k = 1$, -0.014 at $k = 20$. By $k = 10$ the self-vote is only a $1/10$ weight, so the two AUCs estimate nearly the same quantity and the sign of the difference is decided by sampling noise: this particular test set is slightly easier than the training set (it has fewer malignant cases, 36% vs 38%), which is why the test curve ends up above. Nothing about the model generalizes “better than it fits”; the two numbers are simply two noisy estimates once the memorization advantage is gone.
- Small k means high flexibility, low bias, and high variance; the training set rewards the flexibility and the test set pays for the variance. That is the ordering seen in the last panel.

Why training performance misleads. Training predictions reuse the labels being predicted, so they measure how well the method can reproduce data it has already seen, not how well it generalizes. For KNN the effect is extreme

because the model stores the training data; for $k = 1$ the “fit” is the identity map. A new observation is never in its own neighbour set, so the only honest estimate of its error comes from data held out of the fit (a test set or cross-validation), and that is what should be used to choose k . Here that choice is $k = 10$ or 20 , not the $k = 1$ that training AUC would pick.

Question 3 (Peer evaluation of Homework 5)

Solution

Issue #1 in my repository, opened by [rqzhu-aide](#), assigned the Homework 5 reports of [steven-jia-uiuc](#) and [krishay-b](#). I reviewed both. My selected item is from Krishay’s report; a second, shorter review of Steven’s report follows it.

Selected item: Krishay, Question 2(c)

1. Mistake. In the paragraph on the nonlinear terms of f , the report claims that in Setting 2 (pairwise correlation 0.8) the lasso can partly capture the $-0.3X_3^2$ term through the correlated covariates. It cannot; the claim is a conceptual error about what a linear model can represent.

2. Assessment. The report states:

“In Setting 2, correlation also lets lasso leak some of the X_3^2 signal into the shared factor, but only as a linear approximation.”

In Setting 2 the covariate vector is jointly Gaussian with mean zero. For any zero-mean Gaussian vector every third moment is zero, so $\text{Cov}(X_j, X_3^2) = E[X_j X_3^2] = 0$ for every j , including $j = 3$ and the shared-factor direction $\sum_j X_j$. A linear model in the 30 covariates therefore has zero correlation with X_3^2 and cannot capture any of the $-0.3X_3^2$ term, no matter how strong the correlation among the covariates. The report’s own table shows this: the fitted slope on X_3 is 0 in Setting 2, exactly as in Setting 1. If leakage existed, correlation would have made the slope nonzero, not left it at zero.

3. Correction. The sentence should state the opposite: the $-0.3X_3^2$ term is invisible to lasso in *both* settings, so lasso carries a fixed misspecification error of about 0.18 (the variance of $0.3X_3^2$) plus about 0.066 from the nonlinear part of $\sin(X_2)$, in either setting. Correlation changes how lasso spreads weight across proxies for X_1 and X_2 , but does nothing for the squared term. A direct check: regress X_3^2 on all 30 covariates generated as in Setting 2; the R^2 is essentially zero.

```
set.seed(1); n <- 1e5
W <- rnorm(n); U <- matrix(rnorm(n * 30), n)
X <- sqrt(0.8) * W + sqrt(0.2) * U                                # Setting 2: unit variances,
  <- correlation 0.8
c(R2_X3sq_on_X = summary(lm(X[, 3]^2 ~ X))$r.squared, # no linear leakage
  max_abs_cor = max(abs(cor(X, X[, 3]^2))))
#> R2_X3sq_on_X max_abs_cor
#> 0.000479952 0.010116933
```

Posted Issue: <https://github.com/krishay-b/stat432-fall2026/issues/2>

Second review: Steven, Question 2(c)

1. Item for improvement. The answer to part (c) is incomplete: it names the nonlinearity but does not explain how it affects the comparison, and it does not answer whether one method is always better.

2. Assessment. The report’s entire answer to part (c) is:

“Also, the true model is nonlinear in X_2 and X_3 , while this lasso fit only uses linear terms. From these two simulations I cannot say that correlation always helps KNN, only that it helped in this example.”

The question asks two things this does not answer. First, *how* the nonlinearity affects the comparison: the report notes the mismatch but never says what it does to the results. Second, whether the results justify a claim that one method is always better: only the correlation claim is addressed. As written, a reader cannot tell whether lasso lost in Setting 2 because KNN found good neighbours or because lasso was fitting the wrong model.

3. Correction. A linear fit has an irreducible floor here. With independent standard normal covariates, the best linear predictor of $0.5x_1 + \sin(x_2) - 0.3x_3^2$ leaves residual variance of about 0.254: 0.178 from the squared term, 0.066 from the nonlinear part of the sine, and 0.01 noise. Lasso cannot go below that with any sample size, while KNN has no such floor. The comparison therefore mixes neighbourhood quality with model misspecification, and the Setting 2 result (lasso 0.246 vs. 5NN 0.192) is mostly lasso sitting at its floor. No general claim is justified: with a linear f lasso would win both settings, and with a response depending on differences such as $X_1 - X_2$, strong correlation would shrink the relevant signal in the distance and hurt KNN.

```
x <- rnorm(1e6)
c(sine_residual = var(sin(x)) - cov(x, sin(x))^2, squared_term = var(0.3 * x^2),
  floor = var(sin(x)) - cov(x, sin(x))^2 + var(0.3 * x^2) + 0.01)
#> sine_residual  squared_term      floor
#>      0.064393      0.181018      0.255411
```

Posted Issue: <https://github.com/steven-jia-uiuc/stat432-fall2026/issues/2>